Lab 7: Socket Programming

Link State Emulator

An oracle node in Python and virtual nodes in C++

Name	Roll number
Mahant Sakhare	24B0956
Divyaansh Narkhede	24B0981

Contents

1 Files and how to run **2**

2 Overall design **2**

3 Message formats **3**

3.1 TCP framing 3

3.2 CONNECT (VN to ON, TCP) 3

3.3 LINK-STATE (ON to VN, TCP) 3

3.4 LSP (VN to VN, UDP) 4

4 The oracle node (`oracle.py`) **4**

4.1 Reading the config file 4

4.2 Giving out letters 4

4.3 When LINK-STATE is sent 4

4.4 Watching the config file 5

4.5 Main loop 5

5 The virtual node (`vn.cpp`) **5**

5.1 State 5

5.2 Start-up 5

5.3 Main loop 6

5.4 Handling LINK-STATE 6

5.5 Sending and flooding LSPs 6

5.6 Building and printing the graph 7

6 Sockets, blocking and threads **7**

7 Error handling **8**

8 Testing **8**

8.1 On one laptop 8

8.2 On two laptops 8

1 Files and how to run

File	Contents
`oracle.py`	The oracle node, in Python 3. Uses only the standard library (`socket`, `select`, `struct`, `os`).
`vn.cpp`	The virtual node, in C++17. Uses plain POSIX sockets.
`config_file.txt`	The example network from the assignment.

Start the oracle first. It takes the config file and, optionally, the IP address to listen on. If no IP is given, it works out the machine's own address.

```
python3 oracle.py config_file.txt
python3 oracle.py config_file.txt 10.51.3.203
```

Then compile the virtual node (VN) once and start one copy per node. The arguments are the oracle's IP, this node's UDP port and this node's own IP.

```
g++ -std=c++17 -Wall -Wextra -O2 -o vn vn.cpp
./vn 10.51.3.203 6000 10.51.3.203     # first to connect, becomes A
./vn 10.51.3.203 6001 10.51.3.203     # becomes B, and so on
```

The order in which VNs connect decides their letters. If the config describes five nodes and a sixth VN connects, it becomes node F, an island with no neighbours.

2 Overall design

Figure 1 shows the example network from the assignment. The dashed lines are the TCP connections that every VN keeps open to the oracle node (ON). The solid lines are the virtual links. Link state packets (LSPs) only travel along these, as UDP datagrams.

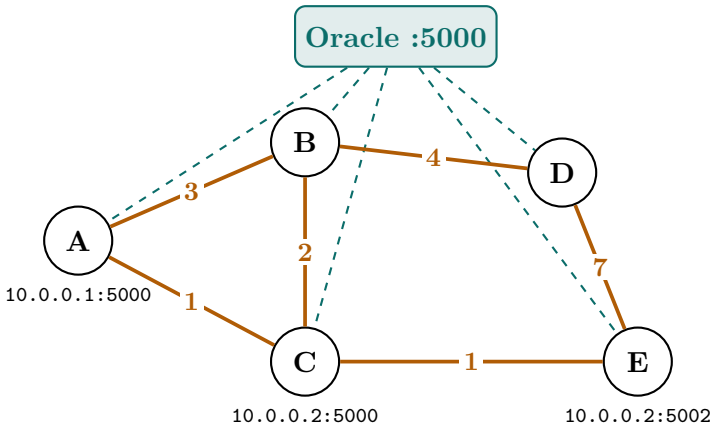


Figure 1: The example network. Dashed teal lines are TCP connections to the oracle. Solid amber lines are virtual links with their costs.

A normal run goes through five steps:

1. The ON reads the config file and listens on TCP port 5000. Each VN binds its UDP port.
2. Each VN connects to the ON and sends a CONNECT message with its IP and UDP port. The ON gives it the next free letter.

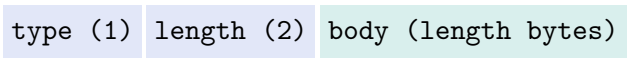
3. When all N nodes from the config have sent CONNECT, the ON sends each VN a LINK-STATE message with its own letter and its neighbours. The VN prints it, for example `E=0,C=1,D=7`.
4. Each VN sends its LSP to its neighbours. A VN that receives a newer LSP stores it and passes it on. Soon every VN holds every LSP.
5. Every 10 seconds each VN sends its LSP again and prints the graph it knows. When the config file changes, the ON sends new LINK-STATE messages and the VNs flood new LSPs straight away.

3 Message formats

All three messages are binary, as the assignment requires. Every number with more than one byte is sent in network byte order (big endian), so the Python side and the C++ side read the same value. Python uses `struct` with the `"!"` prefix and C++ uses `htons` and `htonl`.

3.1 TCP framing

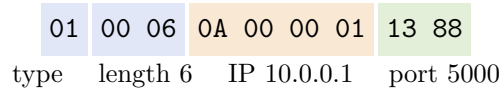
TCP is a stream of bytes and does not keep message boundaries. So every TCP message starts with a 3-byte header: 1 byte for the type and 2 bytes for the length of the body. The receiver reads exactly 3 bytes, then exactly that many more.



The types are 1 for CONNECT and 2 for LINK-STATE. The LSP uses type 3 but travels over UDP, so it needs no length field.

3.2 CONNECT (VN to ON, TCP)

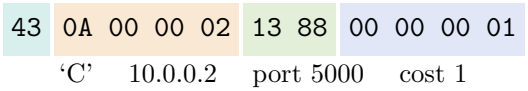
Sent once, right after the TCP connection is made. The body is the VN's IPv4 address (4 bytes) and UDP port (2 bytes). Example for node A at 10.0.0.1, port 5000, 9 bytes in total:



3.3 LINK-STATE (ON to VN, TCP)

The body is a 1-byte count followed by that many 11-byte tuples. Each tuple is (letter, IP, UDP port, cost), with 1, 4, 2 and 4 bytes. Exactly one tuple has cost 0. That tuple names the receiving VN itself, and it is how a VN learns its letter. All other tuples are its neighbours.

Example: the message to node E has three tuples, (C, 10.0.0.2, 5000, 1), (D, 10.0.0.2, 5001, 7) and (E, 10.0.0.2, 5002, 0). The body is $1 + 3 \times 11 = 34$ bytes. One tuple looks like this:



3.4 LSP (VN to VN, UDP)

The LSP carries the origin's letter, a sequence number and the origin's list of (neighbour, cost) pairs. It has no IP addresses, because each VN already knows how to reach its neighbours.

```
type (1) | origin (1) | seq (4) | count (1) | letter (1) | cost (4) ...
```

E's first LSP is 17 bytes: `03 45 00000001 02 43 00000001 44 00000007`. UDP delivers each datagram whole, so the receiver checks that the size is exactly $7 + 5 \times \text{count}$.

Message	Over	When it is sent	Largest size
CONNECT	TCP	Once, after connecting	9 bytes
LINK-STATE	TCP	When all N nodes have joined, after each config change, and to an extra node that joins late	$3 + 1 + 26 \times 11 = 290$
LSP	UDP	After each LINK-STATE, every 10 s, and when passing on a newer LSP. Each send goes out 3 times.	$7 + 26 \times 5 = 137$

Table 1: Summary of the three messages.

4 The oracle node (`oracle.py`)

4.1 Reading the config file

The config file gives only the upper right triangle of the cost matrix, because links are symmetric. Row i lists the costs from node i to every later node, and -1 means there is no link. `parse_config()` skips empty lines and lines starting with `#`, and strips leading tabs.

The number of nodes comes from the first row: four numbers in A's row means five nodes. The function then checks that there are at most 26 nodes, that there are exactly $N - 1$ rows, that row i has $N - 1 - i$ entries, and that every cost is -1 or greater than 0. It fills both `cost[i][j]` and `cost[j][i]`, and puts 0 on the diagonal. A line that is not made of numbers is skipped with a warning.

4.2 Giving out letters

The ON keeps a list `nodes` in connection order. `nodes[0]` is A, `nodes[1]` is B, and so on. Each entry holds the TCP socket, the IP and the UDP port. The IP and port stay empty until CONNECT arrives.

When a new connection arrives, `accept_new()` first looks for a slot whose connection closed before it ever sent CONNECT, and reuses that letter. This stops a stray connection, such as a port scan or a VN that crashed early, from holding letter A forever and blocking the network from starting. Otherwise the connection gets the next letter. If the letter is beyond the nodes in the config, the node is logged as a disconnected island.

4.3 When LINK-STATE is sent

`handle_message()` reads a CONNECT and stores the IP and port. If all N nodes from the config have now sent CONNECT, it sets `started` and sends a LINK-STATE to every node. If the network

has already started, the sender must be a late extra node, so only that node gets a LINK-STATE, holding just its own zero-cost tuple.

`send_link_state(i)` builds the message for one node. For every node j whose address is known, it adds a tuple with cost 0 if $j = i$, or with `cost[i][j]` if that cost is positive. Extra nodes beyond the config never have positive costs, so they only receive their own tuple.

4.4 Watching the config file

On every pass of the main loop, about once a second, `check_config()` reads the file's modification time. If it has changed, the file is parsed again. The new matrix is used only if it is valid, has the same number of nodes, and actually differs from the old one. Then every node gets a fresh LINK-STATE. Saving the file without changing it sends nothing. A broken edit is logged and the old topology is kept.

4.5 Main loop

```
while True:
    socks = [listener] + [nd.sock for nd in nodes if nd.sock is not None]
    readable, _, _ = select.select(socks, [], [], 1.0)
    for s in readable:
        if s is listener:
            accept_new(listener)
        else:
            ...  # find the node that owns s, then handle_message(i)
    check_config()
```

The ON listens on its real network IP, never on 127.0.0.1 or 0.0.0.0. If no IP is given on the command line, `find_my_ip()` connects a UDP socket to 8.8.8.8 and reads the local address with `getsockname()`. A UDP connect sends no packet. It only makes the kernel choose the outgoing interface.

5 The virtual node (`vn.cpp`)

5.1 State

The VN uses only fixed-size global arrays and no dynamic memory, as the assignment suggests. `nbrs[]` holds the neighbours from the latest LINK-STATE (letter, IP, port and cost). `db[]` holds the latest LSP from each origin, indexed by letter: whether it is known, its sequence number, and its list of neighbours and costs. A VN also stores its own LSP in `db`.

5.2 Start-up

1. Check the arguments. Both IPs must be valid and must not be 127.x.x.x or 0.0.0.0, and the port must be between 1024 and 65535.
2. Create the UDP socket and bind it to the VN's own IP and port. On failure, print the reason and exit, as the assignment asks.
3. Connect to the ON on TCP port 5000. If the ON is not running yet, close the socket, wait 2 seconds and try again.
4. Send the 9-byte CONNECT message.

The UDP port is bound before the VN contacts the ON. A VN that cannot get its port therefore exits before it takes up a letter.

5.3 Main loop

Figure 2 shows the VN's event loop. `select()` watches the UDP socket and, while it is still open, the TCP socket to the ON, with a 1 second timeout.

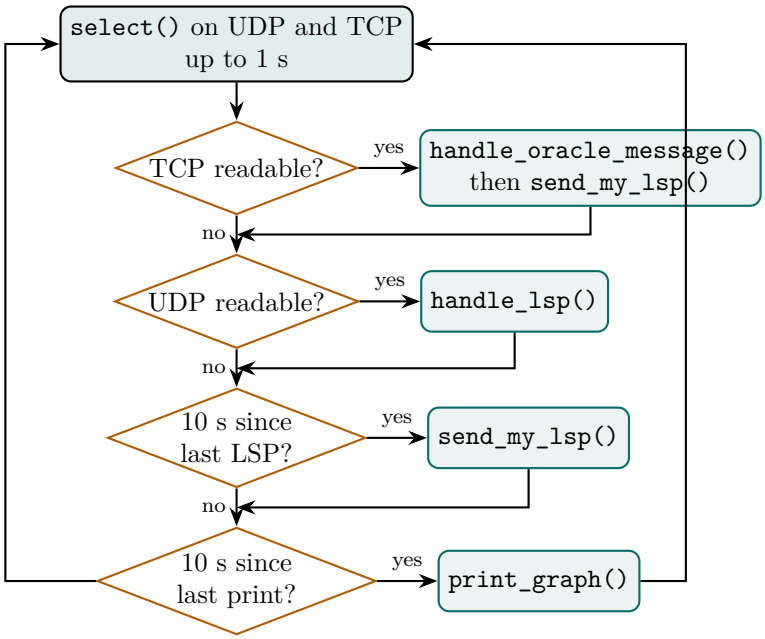


Figure 2: One pass of the VN's event loop. The timed steps only run once the VN knows its letter.

5.4 Handling LINK-STATE

`handle_oracle_message()` reads the 3-byte header and then the body. It checks that the type is LINK-STATE and that the length equals $1 + 11 \times \text{count}$. It then rebuilds `nbrs[]` from the tuples. The tuple with cost 0 gives the VN its letter. Finally it prints the line the assignment asks for, for example:

```
LINK-STATE: E=0,C=1,D=7
```

After a valid LINK-STATE the main loop sends a new LSP at once, so changes spread quickly.

5.5 Sending and flooding LSPs

`send_my_lsp()` adds one to the VN's sequence number, stores the new LSP in its own `db` row, builds the packet and calls `flood()`. `flood()` sends the packet with `sendto()` to every neighbour, three times over. The assignment says LSPs are rarely lost but suggests sending each one two or three times for safety. The extra copies do no harm, because receivers drop them.

`handle_lsp()` reads one datagram with `recvfrom()` and checks every field before using it: the type, an origin letter between A and Z, a size of exactly $7 + 5 \times \text{count}$, and a valid letter in every entry. It then drops the packet if it is the VN's own LSP coming back, or if the VN already has an LSP from that origin with an equal or higher sequence number. Otherwise it stores the LSP and floods it, unchanged, to all its neighbours.

The sequence number check is what makes flooding stop. Each VN passes on a given LSP only the first time it sees it, so the flood dies out even though the graph has cycles. For example, A's first LSP reaches B and C in the first round, D and E in the second, and in the third round every copy is dropped.

LSPs are also sent every 10 seconds. This replaces any LSP that was lost, and it lets a VN that has just gained a neighbour learn the rest of the network.

5.6 Building and printing the graph

Every 10 seconds `print_graph()` asks `build_graph()` for a 26×26 matrix and prints its upper right triangle, in the same layout as the config file:

```
---- [E] known graph (-1 = no link) ----
      B   C   D   E
A:    3   1  -1  -1
B:        2   4  -1
C:           -1   1
D:                7
```

`build_graph()` draws a link between k and j only if both ends report it, which is the usual two-way check in link state protocols:

```
// True if node j's latest LSP lists node k as a neighbour.
bool reports(int j, int k) {
    if(!db[j].known) return false;
    for(int e=0; e<db[j].count; e++) {
        if(db[j].ids[e] == 'A' + k) return true;
    }
    return false;
}
```

Without this check we found a real bug. When a config change removed both of E's links, C and D sent new LSPs without E. E's own new LSP had no neighbour to travel through, though, so every other VN kept E's old LSP listing C and D. Those two links then stayed in everyone's graph forever. With the check, a link that only one side claims is not drawn.

The VN does not compute shortest paths. The assignment only asks each node to learn and print the graph.

6 Sockets, blocking and threads

Both programs are single-threaded and use blocking sockets together with `select()`. `select()` waits on several sockets at once and returns when at least one of them has data, or when the 1 second timeout runs out. We call `recv()`, `recvfrom()` or `accept()` only on a socket that `select()` has reported as readable, so these calls return straight away. The timeout lets each program do its timed work when nothing arrives: the ON checks the config file and the VN checks its 10 second timers.

TCP may split or join messages, so both programs read a TCP message in two steps (header, then body) and loop until they have every byte. In C++ these loops are `send_all()` and `recv_all()`. Python's `sendall()` already loops, and `recv_exact()` does the reading. UDP keeps datagram boundaries, so one `recvfrom()` always returns one whole LSP.

The one place where a call could really block is a TCP message that arrives only in part. The

Program	Socket	Calls	Mode
ON	TCP listener	`socket`, `setsockopt(SO_REUSEADDR)`, `bind`, `listen`, `accept`	blocking, with `select`
ON	TCP, one per VN	`recv` (in `recv_exact`), `sendall`	blocking, with `select`
ON	UDP	not used for messages; one short-lived socket in `find_my_ip`	none
VN	TCP to the ON	`socket`, `connect`, `send` (`MSG_NOSIGNAL`), `recv`	blocking, with `select`
VN	UDP for LSPs	`socket`, `bind`, `sendto`, `recvfrom`	blocking, with `select`

Table 2: Every socket in the two programs. Both programs run in a single thread.

reading loop would then wait for the rest. Our messages are small and each is sent with a single call, so this did not happen in testing.

7 Error handling

Problems at start-up stop the program with a clear message, because nothing useful can run without a valid config, a port or an oracle. Problems while running are logged and skipped, and the program keeps going.

If the ON stops while the VNs are running, each VN prints that it lost the oracle, closes the TCP socket and carries on with its last neighbour list. Config changes no longer reach it, and it does not reconnect.

8 Testing

8.1 On one laptop

We ran the ON and five or six VNs on one laptop, all using the laptop's real Wi-Fi address.

Part of the ON's output:

```
[ON] config has 5 nodes
[ON] listening on TCP 10.51.3.203:5000
[ON] accepted 10.51.3.203:53270 as node A
[ON] CONNECT from A = 10.51.3.203:6000
...
[ON] all 5 nodes connected, sending LINK-STATE messages
[ON] LINK-STATE -> A: A=0,B=3,C=1
[ON] LINK-STATE -> E: C=1,D=7,E=0
[ON] accepted 10.51.3.203:59220 as node F (extra node: disconnected island)
[ON] config file changed
[ON] LINK-STATE -> A: A=0,B=3,C=1,D=2
```

8.2 On two laptops

We also did some testing on two laptops connected to the same hotspot.

	Situation	What happens
ON	Wrong arguments, missing config file, invalid config	Error message and exit
ON	Port 5000 busy, or no network route	Error message and exit, with a hint to pass the IP
ON	Connection closes before CONNECT	Socket closed; letter reused by the next connection
ON	VN closes or crashes later	`recv` returns 0; socket closed and no longer watched
ON	Message that is not a valid CONNECT	Logged and ignored
ON	Sending LINK-STATE fails	That node is marked disconnected
ON	More than 26 connections	Extra connection closed
ON	Config edited into an invalid file, or with a different N	Logged; old topology kept
VN	Bad arguments, loopback IP, port out of range	Error message and exit
VN	UDP port cannot be bound	`perror` message and exit
VN	Oracle not running yet	Retry every 2 seconds
VN	Oracle closes the connection	Stop watching TCP; keep running the LSP protocol
VN	Bad header length from the oracle	Treated as a broken connection
VN	Wrong type or length, or no zero-cost tuple	Logged and ignored
VN	Malformed or oversized UDP datagram	Dropped before anything is stored
VN	Own LSP, or an old or duplicate LSP	Dropped and not passed on
VN	Lost LSP	Covered by the 3 copies and the 10 second resend
VN	Writing to a closed TCP connection	`MSG_NOSIGNAL` gives an error instead of SIGPIPE

Table 3: Error cases and how each program handles them.